

CO5

ASSESSMENT TOOL-1

NAME : P. Ranjithkumar

DATE:19.08.2026

REG.NO: 192411119

Wednesday

COURSE CODE : CSA1103

COURSE NAME : OOAD

Q1. Online Banking System

Introduction

An Online Banking System allows customers to manage accounts, transfer funds, pay bills, and view transaction history. Since banking applications deal with financial and personal information, the system must satisfy strict **security, performance, reliability, and accuracy constraints**.

The given constraints are:

1. Transaction completion time must be **less than 2 seconds**.
2. **Multi-factor authentication (MFA)** is mandatory.
3. The system must support **5000+ concurrent users**.

System Components

- Account Management
- Fund Transfer
- Bill Payment
- Transaction History

Quality Assurance Analysis

Quality assurance ensures that the system meets both functional and non-functional requirements. Testing should begin from individual modules and continue through integration, system, security, and performance testing.

Potential Security Defects

1. Weak password validation may allow unauthorized login.
2. MFA may be incorrectly implemented or bypassed.
3. Session IDs may be exposed.
4. Unauthorized users may access another customer's account.
5. Sensitive banking information may be transmitted without adequate protection.
6. Improper authorization may allow users to transfer money without permission.
7. Multiple failed login attempts may not be properly restricted.

Potential Performance Defects

1. Fund transfer may take more than 2 seconds.
2. Database queries may become slow when thousands of users access the system.
3. Server response may become slow during peak periods.
4. Payment processing may result in timeout errors.
5. Excessive database connections may overload the server.

Potential Reliability Defects

- Money deducted but transaction status remains pending.
- Money transferred twice because of repeated requests.
- Transaction history not updated.
- System failure during fund transfer.
- Incorrect account balance after transaction.
- Data loss after server/database failure.

Testing Strategies

1. Functional Testing

Verify:

- Login and logout.
- Account balance.
- Fund transfer.
- Bill payment.

- Transaction history.
- Transaction confirmation.

2. Security Testing

Test:

- Password security.
- MFA.
- Authorization.
- Session management.
- Access control.
- Unauthorized fund transfers.

3. Performance Testing

Measure whether transactions complete in less than 2 seconds under normal conditions.

4. Load Testing

Simulate **5000+ users** performing banking operations simultaneously.

5. Stress Testing

Gradually increase users beyond 5000 to identify the system's breaking point.

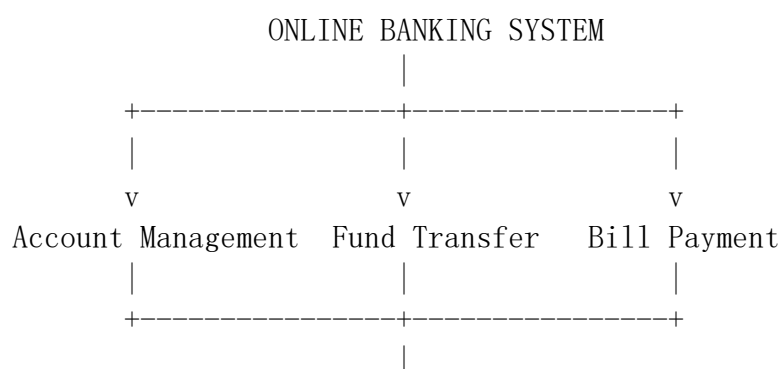
6. Reliability Testing

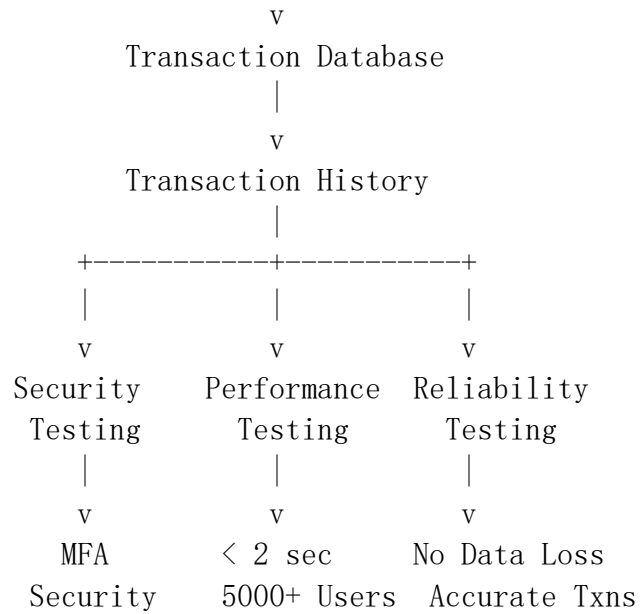
Repeatedly execute transactions and introduce failures such as network interruption or database failure.

7. Recovery Testing

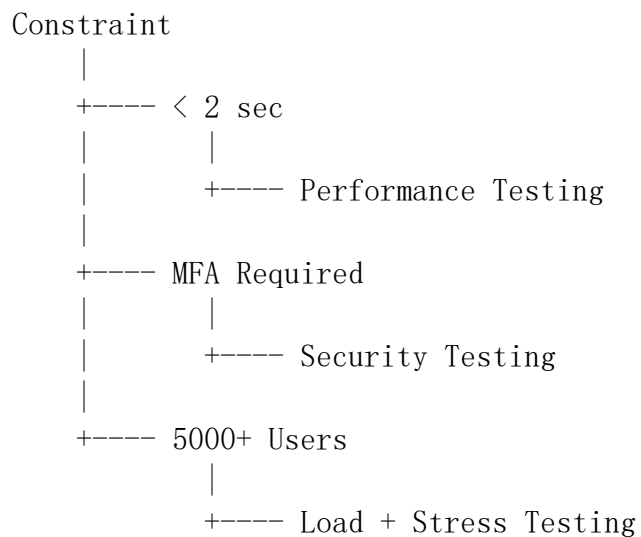
Verify that the system can recover transactions correctly after crashes or service failures.

Diagram





Constraint-Based Testing



Expected Quality

The system should provide:

- Secure authentication.
- Fast transactions.
- Accurate balances.
- No duplicate transactions.
- High availability.
- Reliable transaction records.

Conclusion

The quality of an online banking system depends on satisfying security, performance, and reliability constraints simultaneously. Security testing verifies MFA and access control, performance testing ensures transactions complete within **2 seconds**, and load testing confirms support for **5000+ concurrent users**. Reliability and recovery testing ensure that financial transactions are not lost or duplicated. Therefore, a combination of functional, security, performance, and reliability testing is essential for delivering a trustworthy banking system.

Q2. E-Commerce Shopping Application

Introduction

An E-Commerce Shopping Application allows customers to search for products, add products to a shopping cart, and complete purchases using a payment gateway. Since customers interact directly with the application, **usability, simplicity, accessibility, and responsiveness** are important quality factors.

Given Features

- Product Search
- Shopping Cart
- Checkout
- Payment Gateway

Usability Constraints

1. Checkout must be completed within **3 steps**.
2. Product navigation should be easy.
3. The application should be responsive on both **mobile and desktop**.

Usability Evaluation

Usability testing evaluates how easily users can perform important tasks without confusion or assistance.

1. User Testing

Real users are asked to perform tasks such as:

- Search for a product.
- Apply a filter.
- Add a product to cart.
- Change quantity.
- Complete checkout.

The tester observes:

- Time taken.
- Number of errors.
- Number of clicks.
- User confusion.
- Task completion rate.

2. Task-Based Testing

Example:

Task: Purchase a Laptop

Open Website



Search Laptop



Select Product



Add to Cart



Checkout



Payment



Order Confirmation

The objective is to determine whether users can complete the task quickly and correctly.

3. Accessibility Testing

Check:

- Font size.
- Color contrast.
- Keyboard navigation.

- Screen reader compatibility.
- Button size.
- Clear labels.
- Alternative text for images.

4. Responsive Testing

The application should be tested on:

- Mobile phones.
- Tablets.
- Laptops.
- Desktop monitors.

Usability Challenges

Navigation Problems

Users may have difficulty finding categories and filters.

Complex Checkout

Too many pages may increase cart abandonment.

Poor Mobile Design

Small buttons and incorrectly aligned fields can cause errors.

Unclear Error Messages

Messages such as "Invalid input" do not tell users what needs to be corrected.

Too Much Information

Excessive advertisements and unnecessary information can distract customers.

Recommended Improvements

1. Three-Step Checkout

STEP 1

Address



STEP 2

Payment



STEP 3

Confirmation

2. Better Product Navigation

Provide:

- Search bar.
- Categories.
- Filters.
- Sorting.
- Product recommendations.
- Breadcrumb navigation.

3. Mobile-Friendly Design

Use:

- Responsive layouts.
- Large buttons.
- Touch-friendly controls.
- Properly sized images.
- Simplified menus.

4. Error Prevention

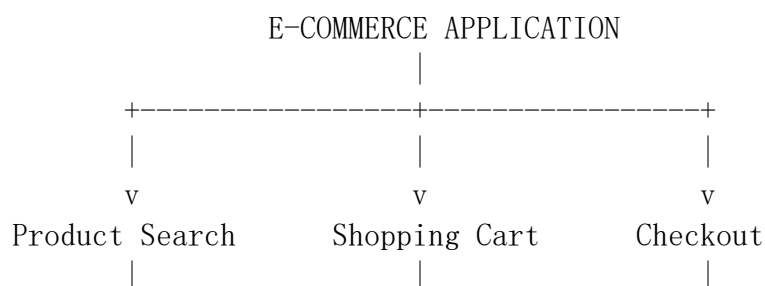
Instead of showing:

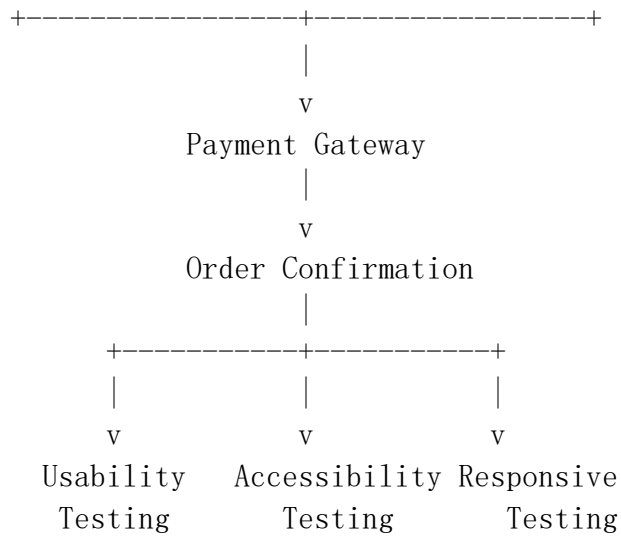
Invalid Input

show:

Please enter a valid 10-digit mobile number.

Diagram





Customer Satisfaction Improvement

The proposed improvements:

- Reduce checkout time.
- Reduce user errors.
- Make products easier to find.
- Improve mobile shopping.
- Reduce confusion.
- Increase successful purchases.

Conclusion

Usability testing helps identify difficulties that customers face while searching and purchasing products. A three-step checkout, simple navigation, responsive design, clear error messages, and accessible controls make the application easier to use. These improvements reduce customer effort, improve shopping efficiency, reduce cart abandonment, and increase customer satisfaction.

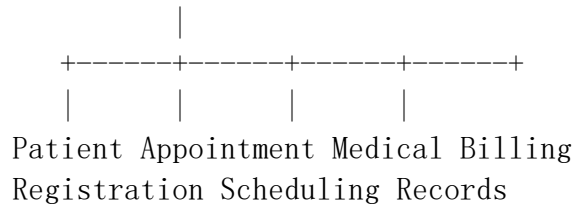
Q3. Hospital Management System

Introduction

A Hospital Management System manages critical healthcare operations including patient registration, appointment scheduling, medical records, and billing. Incorrect patient information or system downtime can seriously affect hospital operations. Therefore, QA must give high priority to **accuracy, availability, security, concurrency, and reliability**.

Modules

Hospital Management System



QA Constraints

- Patient data accuracy must be maintained.
- System should be available 24/7.
- Multiple departments must access the system simultaneously.

Critical Test Cases

Patient Registration

Test:

1. Valid patient information.
2. Missing mandatory fields.
3. Invalid phone number.
4. Duplicate patient registration.
5. Correct patient ID generation.
6. Correct storage of patient information.

Appointment Scheduling

Test:

1. Appointment with available doctor.
2. Appointment with unavailable doctor.
3. Duplicate appointment.
4. Cancellation.
5. Rescheduling.
6. Simultaneous booking attempts.

Medical Records

Test:

- Correct patient record retrieval.
- Updating medical information.
- Unauthorized access.
- Incorrect patient record selection.
- Data consistency after updates.

Billing

Test:

- Correct bill calculation.
- Multiple services.
- Discounts.
- Payment confirmation.
- Failed payment.
- Duplicate payment.

Important Testing Techniques

Functional Testing

Ensures all hospital functions work according to requirements.

Data Validation Testing

Ensures patient details are accurate and complete.

Security Testing

Prevents unauthorized users from viewing or changing sensitive records.

Concurrency Testing

Checks whether multiple hospital departments can use the system simultaneously without corrupting data.

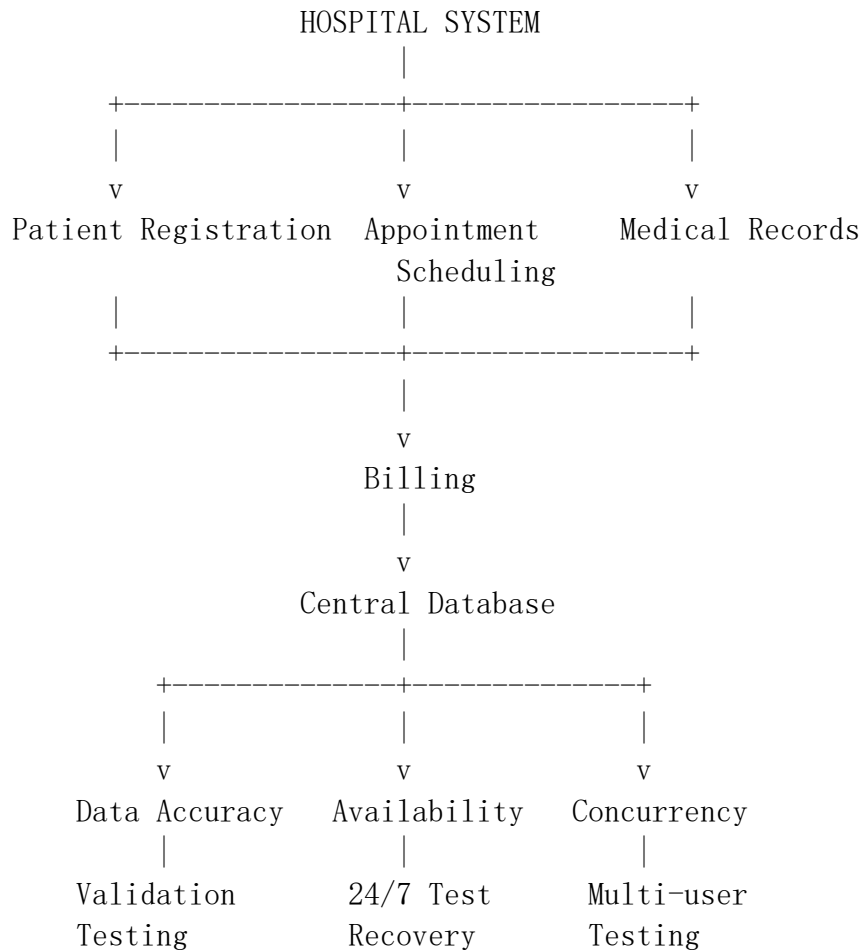
Availability Testing

Checks whether the system remains operational continuously.

Backup and Recovery Testing

Verifies that patient records can be recovered after system or database failure.

Diagram



Multiple Department Access

Reception ----\
Doctor -----\
Laboratory ----- > Central Database
Pharmacy -----/
Billing -----/

All departments may access the same database simultaneously. Therefore, concurrency controls and transaction management are necessary.

Defect Prevention

Defects can be prevented through:

- Input validation.

- Database constraints.
- Duplicate detection.
- Access control.
- Code reviews.
- Automated testing.
- Continuous integration.
- Backup mechanisms.

Testing Priority

Highest Priority



Patient Data Accuracy



System Availability



Security



Concurrency



Performance



Usability

Conclusion

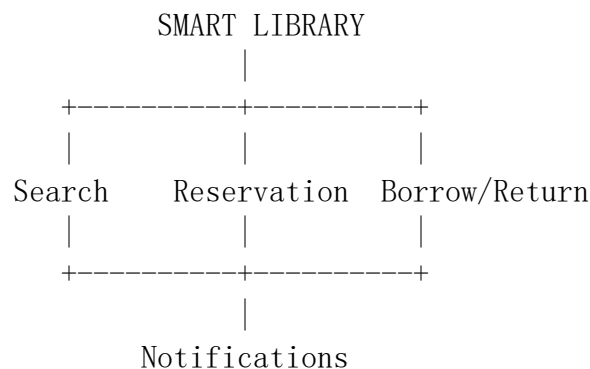
The hospital system requires rigorous QA because it handles critical patient information. Data validation maintains accuracy, availability testing ensures 24/7 operation, and concurrency testing prevents conflicts when multiple departments access records. Backup, recovery, security, and integration testing further improve system reliability. Therefore, constraint-driven testing helps create a safe, accurate, and dependable hospital management system.

Q4. Smart Library Management System

Introduction

A Smart Library Management System helps users search for books, reserve books, borrow and return books, and receive notifications. The major goal of usability testing is to make the system simple enough for **first-time users and users from different age groups**.

Features



Usability Constraints

1. Search results must appear within **2 seconds**.
2. Interface must be accessible to all age groups.
3. First-time users should require minimal training.

Usability Testing Methods

1. User Testing

Users from different age groups perform tasks.

Example:

"Find a Java programming book and reserve it."

Measure:

- Completion time.
- Number of mistakes.
- Number of clicks.
- User satisfaction.

2. Heuristic Evaluation

Experts evaluate:

- Simplicity.
- Consistency.
- Visibility.
- Error prevention.
- Navigation.
- Feedback.

3. Accessibility Testing

Check:

- Font size.
- Contrast.
- Keyboard support.
- Screen-reader support.
- Clear labels.
- Large buttons.

4. Performance Testing

Search requests should return results within **2 seconds**.

Usability Problems

- Complicated menu structure.
- Small fonts.
- Poor contrast.
- Difficult-to-understand icons.
- Too many search fields.
- No clear indication of book availability.
- Complicated reservation process.

Recommended Interface

+-----+ SMART LIBRARY +-----+	
Search for books...	[SEARCH]

Category	Author	Year	Available

Book Name	Available		[RESERVE]
Book Name	Borrowed		[NOTIFY]

Improved User Flow

Login

|

v

Search Book

|

v

Results < 2 Seconds

|

v

Select Book

|

v

Check Availability

|

+---- Available ----> Reserve/Borrow

|

+---- Unavailable --> Notification

Accessibility Improvements

- Adjustable font size.
- High-contrast mode.
- Keyboard navigation.
- Clear textual labels.
- Simple icons.
- Voice/search assistance where appropriate.
- Consistent navigation.

Effect on Usability

Improvement

Benefit

Fast search

Saves time

Improvement	Benefit
Large readable text	Helps elderly users
Simple menus	Reduces learning time
Clear buttons	Reduces errors
Filters	Faster book discovery
Accessibility features	Supports more users

Conclusion

The Smart Library System should prioritize speed, simplicity, accessibility, and clear navigation. Search optimization ensures results appear within **2 seconds**, while readable fonts, high contrast, clear labels, and simple navigation support different age groups. Usability testing with real users helps identify problems before deployment and creates a system with a minimal learning curve.

Q5. Airline Reservation System

Introduction

An Airline Reservation System is used for flight searching, seat selection, ticket booking, and user authentication. Because many users may attempt to book flights simultaneously, the system must provide **high reliability, strong performance, data consistency, and excellent usability**.

Modules

- User Login
- Flight Search
- Seat Selection
- Ticket Booking

Constraints

1. No duplicate booking.
2. System must support peak holiday traffic.
3. Booking interface must be simple and error-free.

Major Risks

1. Duplicate Booking

Two users may attempt to book the same seat simultaneously.

2. Payment Failure

Payment may succeed but ticket generation may fail.

3. High Traffic

Large numbers of users during holidays can cause:

- Slow response.
- Server overload.
- Timeout.
- Database overload.

4. User Errors

Users may enter:

- Wrong passenger name.
- Wrong date.
- Wrong flight.
- Incorrect seat.

QA Testing

Functional Testing

Verify:

- Login.
- Flight search.
- Seat selection.
- Booking.
- Ticket generation.

Security Testing

Test:

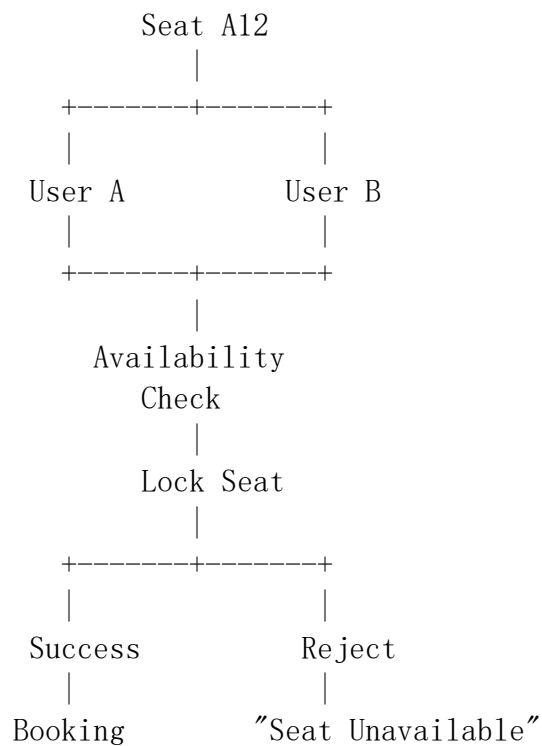
- Authentication.
- Authorization.

- Session management.
- Protection of passenger information.
- Secure payment processing.

Concurrency Testing

Simulate multiple users selecting the same seat.

Example:



Only one user should successfully obtain the seat.

Load Testing

For holiday traffic:

Normal Users

|

v

10,000 Users

|

v

25,000 Users

|

v

50,000+ Users

|

v

Measure:

Response Time

CPU

Memory

Database Load

Error Rate

The exact user capacity should be determined from the system's requirements and infrastructure rather than assumed.

Usability Testing

Users should be able to complete:

Login

↓

Search Flight

↓

Select Flight

↓

Select Seat

↓

Enter Passenger Details

↓

Payment

↓

Ticket Confirmation

Usability Improvements

1. Simple Interface

Use clear buttons such as:

- Search Flight
- Select Seat
- Continue
- Pay Now
- Download Ticket

2. Error Prevention

Before payment, display:

Flight: Hyderabad → Delhi

Date: 20 August

Passenger: 1

Seat: 12A

Total: ₹XXXX

[Confirm & Pay]

3. Clear Error Messages

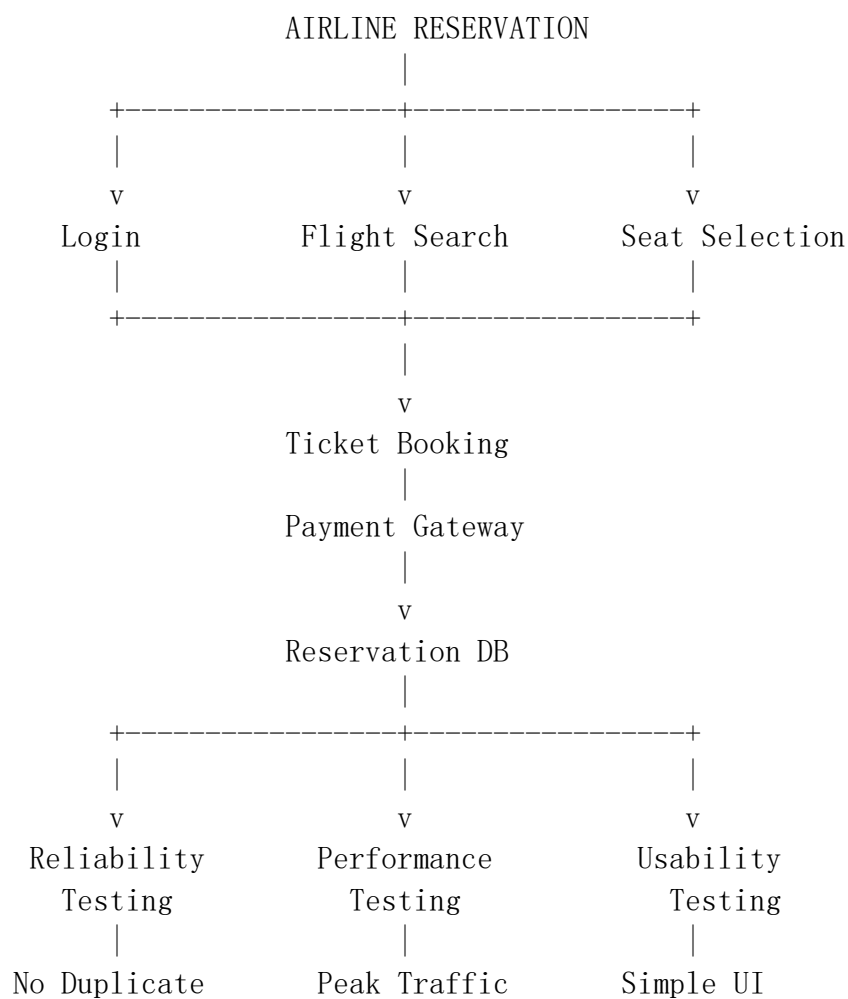
Instead of:

Error 402

Display:

Payment could not be completed. Please try again.

Overall Architecture Diagram



Optimization Techniques

Reliability

- Database transactions.
- Seat locking.
- Duplicate transaction detection.
- Backup and recovery.
- Failure handling.

Performance

- Load balancing.
- Database indexing.
- Caching for frequently searched flight data.
- Efficient database queries.
- Load and stress testing.

Usability

- Simple navigation.
- Fewer unnecessary steps.
- Clear seat map.
- Inline validation.
- Responsive design.
- Clear booking confirmation.

Conclusion

The Airline Reservation System requires both QA and usability testing because failures can directly affect customers and airline operations. Concurrency testing prevents duplicate seat bookings, load testing verifies behavior during peak traffic, and functional testing ensures correct booking operations. Usability testing makes the booking process simple and reduces user errors. Combining these techniques results in a **reliable, high-performance, secure, and customer-friendly reservation system.**